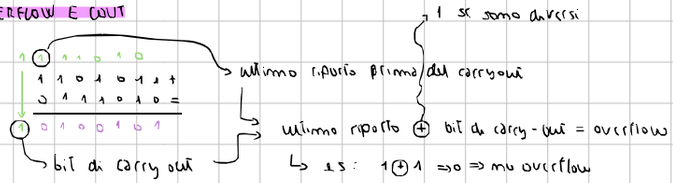
OVERFLOW E COUT



Per operazioni SENZA segno (non in complemento a 2 o a molte) il bit di valle 1 se il risultato è errato è il carry-out

Per operazioni CON segno (in complemento a 2 o a molte) il bit di valle 1 se il risultato è errato è l'overflow (ultimo riporto ⊕ bit di carry - out)

SINTESI SP ⟶ SINTESI A NAND

$F = a \cdot b + c' \cdot d + c \cdot f' + y$

$F = (a \uparrow b)' + (c' \uparrow d)' + (c \uparrow f')' + y$

$F = ((a \uparrow b) \cdot (c' \uparrow d) \cdot (c \uparrow f')\ y')'$

$F = (a \uparrow b) \uparrow (c \uparrow d) \uparrow (c \uparrow f') \uparrow y'$

XOR ⟶ NAND

Definizione di XOR: $a \oplus b = a'b + !ab$

$F = ab' + a'b \equiv F = a \oplus b$ ↓

$F = ab' + a'b + a'a + b'b$ ↓ raccolgo

$F = a(b' + a') + b(a' + b')$ ↓ $b' + a' = (b \cdot a)'$

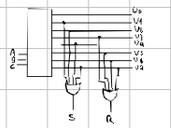
$F = a(b \cdot a)' + b(a \cdot b)'$ ↓ def di ↑

$F = (a \uparrow (b \uparrow a)) \uparrow (b \uparrow (a \uparrow b))$

SINTESI A DECODER

$S = \underset{m_1}{!A!BC} + \underset{m_3}{!AB!C} + \underset{m_5}{A!B!C} + \underset{m_7}{ABC}$

$R = \underset{m_1}{!A!BC} + \underset{m_3}{A!BC} + \underset{m_5}{AB!C} + \underset{m_7}{ABC}$



dopodiché si fa l'OR tra mintermini (AND tra maxtermini nel caso di sintesi PS)

LEGGE DI DE MORGAN

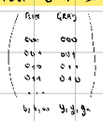
$!(x+y) = !x \cdot !y \qquad !(x \cdot y) = !x + !y$

T DI SHANNON

$A = x \cdot A + !x \cdot A \qquad A = (x+A) \cdot (!x+A)$ ← A valore fisso (0 o 1)

RETE BINARIO - GRAY

Si può ottenere tramite sintesi della tabella binario - gray di (3 bit) per il bit significativo g_m: $g_m = b_m \oplus b_{m+1}$



NOR: OR negato ⇒ vale 1 solo quando gli ingressi sono tutti 0 ⇒D○—

NAND: AND negato ⇒ vale 0 solo quando gli ingressi sono tutti 1 ⇒D○—

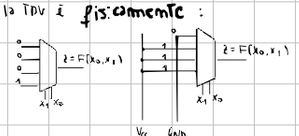
SINTESI A MUX

Voglio realizzare un AND. Dunque la TDV è fisicamente:

x_1	x_0	z
0	0	F(0,0)
0	1	F(0,1)
1	0	F(1,0)
1	1	F(1,1)

⇒

x_1	x_0	z
0	0	0
0	1	0
1	0	0
1	1	1



MASSIMA FREQUENZA

$f = \frac{1}{T_{min}} \qquad T_{min} = T^{FFD}_{out\,max} + T_{max} + T^{FFD}_{set\,max}$

definizione di ↑ (è un AND (-) negato)

II° l. di De Morgan ↳ $!(a+b) = !a\,!b$ NAND = NOT AND ⇒ $(!A\,!B)$

definizione di ↑ (è un AND (-) negato)

ESEMPIO

Y_1:

y_1y_0 \ ab	00	01	11	10
00	0	(-)	0	0
01	0	0	0	(1)
11	(1)	0	(1)	(1)
10	(1)	0	(1)	-

$Y_1 = y_1$ se $ab = 00$

$Y_1 = \overline{y_0}$ se $ab = 01$

$Y_1 = y_1$ se $ab = 11$

$Y_1 = y_0$ se $ab = 10$

Y_0:

y_1y_0 \ ab	00	01	11	10
00	0	-	0	1
01	1	0	1	1
11	1	1	1	0
10	0	1	-	-

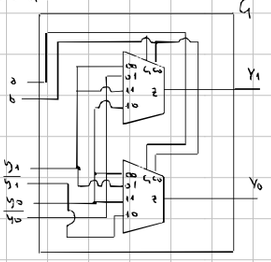
$Y_0 = y_0$ se $ab = 00$

$Y_0 = y_1$ se $ab = 01$

$Y_0 = y_0$ se $ab = 11$

$Y_0 = \overline{y_1}$ se $ab = 10$

Rete G:



SINTESI PS ⟶ SINTESI A NOR

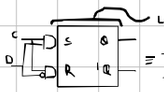
$F = (a' \downarrow b' \downarrow c') \cdot (a' \downarrow c') \cdot f' \cdot y$

$F = [a' \downarrow b' \downarrow c')' \downarrow (a' \downarrow c')' \downarrow f' \downarrow y']'$

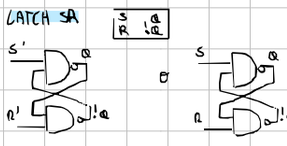
$F = ((a' \downarrow b' \downarrow c') \downarrow (a' \downarrow c') \downarrow f \downarrow y')'$

$F = (a' \downarrow b' \downarrow c') \downarrow (a' \downarrow c) \downarrow f \downarrow y'$

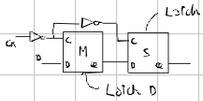
LATCH SR



LATCH SA

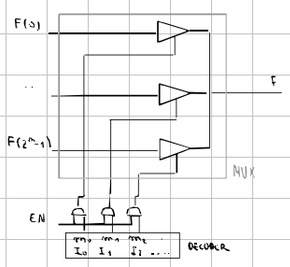


FFD

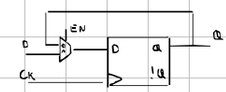


Ad ogni combinazione di ingresso (eccetto !A!B!C non rilevante) corrisponde un mintermine ⇒ il DEC può essere visto come generatore di mintermini: a una specifica configurazione d'ingresso attivo una specifica configurazione di uscita, che corrisponde a un mintermine.

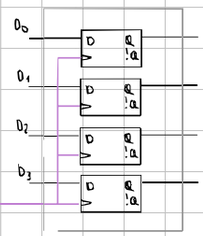
MUX DI BUFFER CON ENABLE



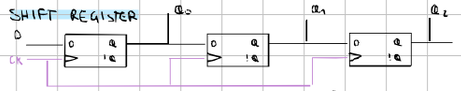
FFD CON ENABLE



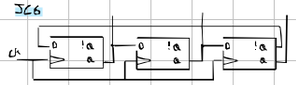
REGISTRO EDGE-TRIGGERED



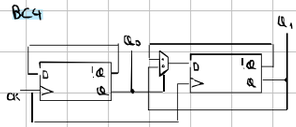
SHIFT REGISTER



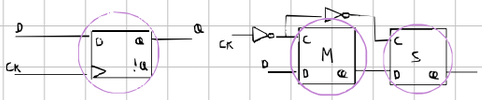
JCG



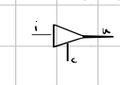
BCG



FFD CON RESET ASINCRONO



BUFFER TRISTATE



SHANNON	NAND	NOR
!A	A ↑ A	A ↓ A
AB	(A ↑ B) ↑ (A ↑ B)	(A ↓ A) ↓ (B ↓ B)
A+B	(A ↑ A) ↑ (B ↑ B)	(A ↓ B) ↓ (B ↓ B)

I circuiti anti-alee consentono di eliminare le alee statiche solo se gli ingressi cambiano una alla volta

Ricordo le frasi del tipo "l'uscita cambia APPENA cambia l'ingresso" o simili indicano la necessità di una rete di Mealy

CODIFICA GRAY A 3 BIT

000
001
011
010
110
111
101
100

Capire a cosa servono gli adder nella sintesi diretta

Guardo l'esempio del semaforo e perché è necessario usare il decoder (bisogna fornire accesso per 3 semafori il rosso, il giallo, ...)

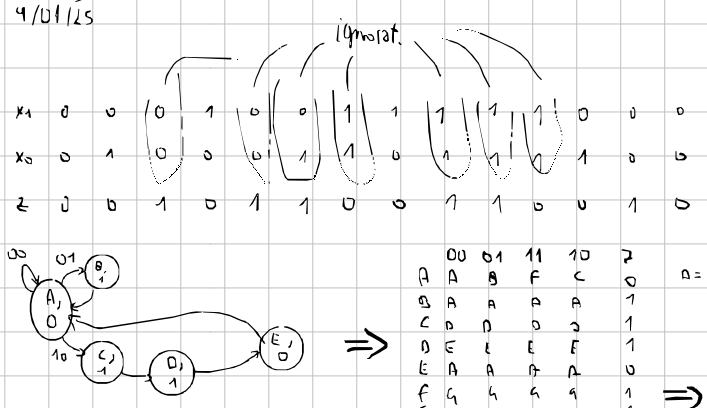
GUARDO IL PROGETTO ESEMPIO DEL SOMMATORE SEQUENZIALE

GUARDO LE MOLTIPLICAZIONI DEI CONTATORI

Capisco perché può dare "-1"

STUDIATI IL COMPARATORE (BIT A BIT?)

4/01/25



⇒

	00	01	11	10	Z
A	A	B	F	C	0
B	A	A	A	A	1
C	D	D	D	D	1
D	E	E	E	E	1
E	A	A	A	A	0
F	G	G	G	G	1
G	H	H	H	H	1
H	I	I	I	I	1
I	E	E	E	E	0

A = 000 B = 001 C = 011 D = 010 E = 110

⇑

	00	01	11	10	Z
A	A	B	–	C	0
B	A	A	–	A	1
C	D	D	–	D	1
D	E	E	–	E	1
E	A	A	–	A	0

⇑

x_1x_0 / $y_2y_1y_0$	00	01	11	10	Z
000	000	001	–	011	0
001	000	000	–	000	1
011	010	010	–	010	1
010	110	110	–	110	1
110	000	000	–	000	0

⇑

Y_2

y_1y_0 / x_1x_0	00	01	11	10
00	0	0	–	0
01	0	0	–	0
11	0	0	–	0
10	1	1	–	1

$y_2 = 0$

y_1y_0 / x_1x_0	00	01	11	10
00	–	–	–	–
01	–	–	–	–
11	–	–	–	–
10	0	0	–	0

$y_2 = 1$

⇓

Dovrebbe essere $Y_2 = y_1 \bar{y}_0$

Z / y_1y_0	y_2 = 0	1
00	0	–
01	1	–
11	1	–
10	1	0

⇒ $Z = y_0 + \bar{y}_2 y_1$

Sintesi: minimo ⇒ raggruppamenti massimi

SINTESI A MUX DI UN FULL-ADDER

a	b	c	S	r
0	0	0	0	0
0	0	1	1	0
0	1	1	0	1
0	1	0	1	0
1	1	0	0	1
1	1	1	1	1
1	0	1	0	1
1	0	0	1	0

PER S (poi R è analogo)

ab / c	0	1
00	0	1
01	1	0
11	0	1
10	1	0

c=0 $S = \bar{a}b + a\bar{b}$

c=1 $S = \bar{a}\bar{b} + ab$

⇒

